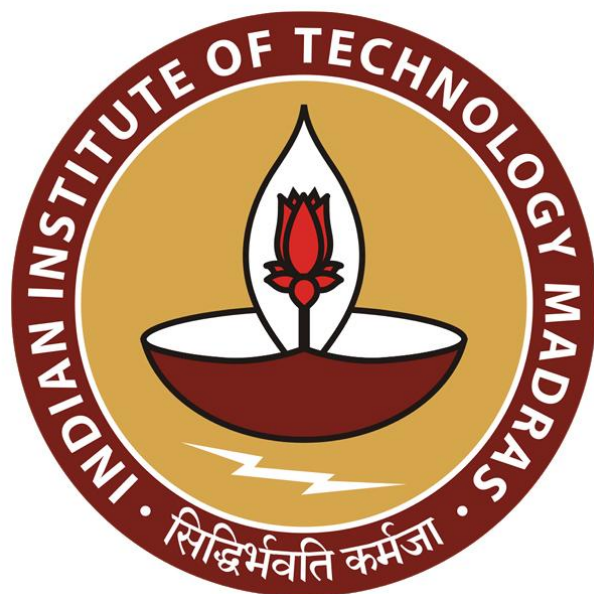


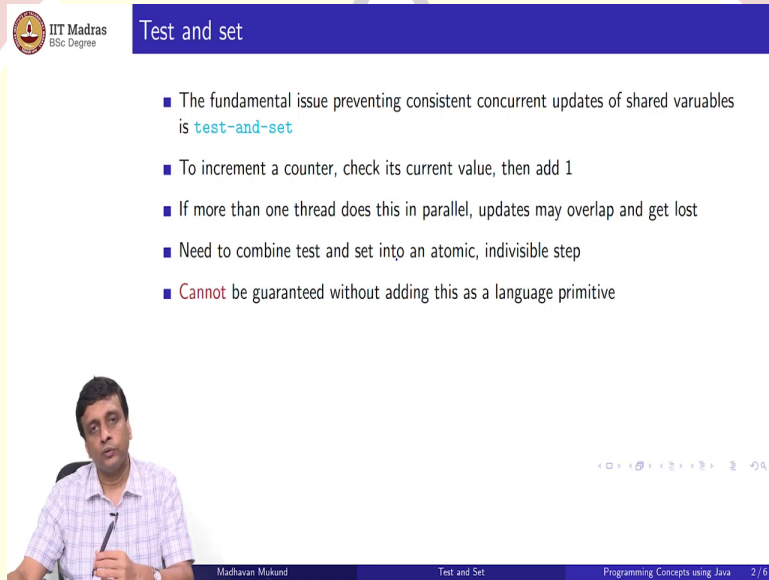


IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Indian Institute of Technology, Madras
Week 10
Test and Set

So we are looking at synchronization of access to critical sections in concurrent programs. And we said that we have these protocols for mutual exclusion, which are tricky to use. So, we would like some programming level support.

(Refer Slide Time: 00:28)



Test and set

- The fundamental issue preventing consistent concurrent updates of shared variables is **test-and-set**
- To increment a counter, check its current value, then add 1
- If more than one thread does this in parallel, updates may overlap and get lost
- Need to combine test and set into an atomic, indivisible step
- **Cannot** be guaranteed without adding this as a language primitive

Madhavan Mukund Test and Set Programming Concepts using Java 2 / 6

So, the main issue that underlies all these race conditions is something called test and set. So, we want to look up a variable. And depending on what that variable says, we want to do something. And between looking up and doing something between testing and setting, we cannot guarantee that something does not interfere. So, the value that we thought that we saw may not be the value that we are actually working with.

So, we looked at this very simple example where two threads were trying to increment an integer, we think of it as a counter or Lamport bakery example. So, what we are really doing in that case is we are testing in the sense that we are checking what is the current value of the integer, then we are computing $n + 1$ and storing it back. So, that is set.

And then this thing again, we said that because we if the two threads test the value at the same time, they see the same old value. And they put back the same new value and then we get a lost update. So, therefore, if you have parallel updates in this test and set regime, either they may see inconsistent values, that is they make an update based on a value that they tested, which is no longer true or some of these updates may get lost because they are both updating the same tested value without realizing that two upgrades are happening in parallel.

So, one way to provide I mean, one essential step is that so, we saw these mutual exclusion protocols which site which bypass this whole question of test and set and created a clever way of doing mutual exclusion of critical sections without using test and set. So, we use shared variables like turn request one request to which did not have this test and set property there is no assumption made about those variables, but by cleverly synchronizing them and how they interact with each other, we are able to ensure that two threads do not simultaneously access the critical section.

And we said that that is error prone. And therefore, we would prefer if the language would actually give us some support and the form in which the language gives us a support is to promise us a test and set operation which is atomic. So, this is something which is guaranteed to us, it is important, it is a promise. So, the language gives you a test and set it is a atomic test and set, where you are promised that nothing is allowed to come in.

So, we are rarely given something which is not available directly, but which has to be given to us as an additional feature of the language. So, you can also argue that you cannot do anything without it. So, test and set is a kind of minimum that you require in order to implement any kind of synchronization in a sensible way at a language level.

(Refer Slide Time: 03:06)



Semaphores

- Programming language support for mutual exclusion
- Dijkstra's **semaphores**
 - Integer variable with atomic test-and-set operation
- A semaphore **S** supports two atomic operations
 - **P(s)** — from Dutch **passeren**, to pass
 - **V(s)** — from Dutch **vrygeven**, to release

■ **P(S)** atomically executes the following

```
if (S > 0) test  
    decrement S; set  
else  
    wait for S to become positive; wait
```



Madhavan Mukund Test and Set Programming Concepts using Java 3/6



Semaphores

- Programming language support for mutual exclusion
- Dijkstra's **semaphores**
 - Integer variable with atomic test-and-set operation
- A semaphore **S** supports two atomic operations
 - **P(s)** — from Dutch **passeren**, to pass
 - **V(s)** — from Dutch **vrygeven**, to release

■ **P(S)** atomically executes the following

```
if (S > 0)  
    decrement S;  
else  
    wait for S to become positive;
```

■ **V(S)** atomically executes the following

```
if (there are threads waiting  
    for S to become positive)  
    wake one of them up;  
    //choice is nondeterministic  
else  
    increment S;
```

$S=0/1$



Madhavan Mukund Test and Set Programming Concepts using Java 3/6

So the first such proposal to add feature, a feature to a language, which supports this kind of synchronization came from Dijkstra, the same Dijkstra who designed the shortest path algorithm for graphs. So, he was very interested in multi processors and in early days of operating systems with multiple threads running, so, he came up with this idea of a semaphore. So, semaphore if you do not know it is like a flag, it is used to be used for signaling and railways and ships and so on. So, you have basically a thing which is, you have a tower with a flag, and you can put it down or you can put it up.

So, this is a semaphore was used for signaling traditionally, in long distance signaling. So, you could see if somebody has put it up, put it down and so on. So, he called these this solution a semaphore. So, semaphore is essentially an integer, but it is an integer with this test and set given to you as a guarantee. So, the test and set operations are called somewhat peculiarly, P and V.

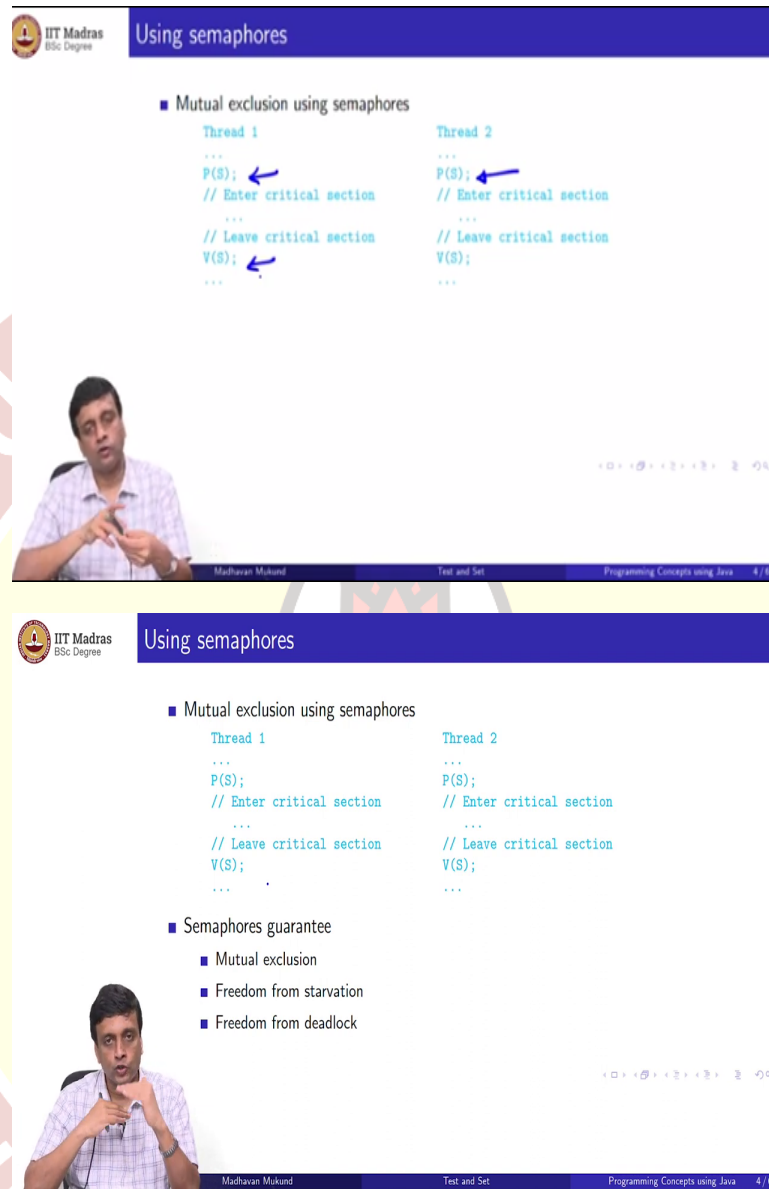
So, these come from two Dutch words *passeren* and *vrygeven*, but essentially, P is used to gain access and V is used to release. So, this is how we should think about it. So, when we do P, we check if the, so, it is with respect to a value s , So, semaphore s , so, it checks whether the current value of the semaphore is above 0. And if it is above 0, then you decrement it and if it is not above 0, then you wait for it to become above 0.

So, this is the test and set so, this is a complicated thing, you are checking it. So, this is a test. And this is a set and there is a wait. So, the wait basically says that you are blocked. So, this is like that busy wait that we saw in Peterson's algorithm and learned, your while something happens, you are just checking it. So, actually, the way that this works is that there is not a busy wait in that sense, because it is part of the programming language. So, you are guaranteed that you will be told when this thing becomes a correct. So, that is what P(S) does.

And V(S) is symmetric. So, what it does is now you have gotten, so you have got this variable which is, so normally I should say that S is usually 0 or 1. So, if you have two processes, S would be normally 0 or 1. So, you see it as 1, and you decrement it. So, you get access, you decrement it so now as becomes 0. So, now the next person who comes is going to see S 0, so they are going to be waiting.

And the next person who comes is also going to see as 0, so a number of people are going to be waiting, not just any one person, So what you will do is that you will say I am done, so you will make one of them up, you do not care which one, you will make one of them up, and then you will increment this and go out. So, that person wakes up. Meanwhile, you have incremented it so that person will now see that the value is positive. So, they will now come here and they will basically go back to this situation and decrement and continue. So, this is how this P and V works.

(Refer Slide Time: 06:23)



Using semaphores

■ Mutual exclusion using semaphores

```
Thread 1
...
P(S); // Enter critical section
...
// Leave critical section
V(S);
...

Thread 2
...
P(S); // Enter critical section
...
// Leave critical section
V(S);
...
```

■ Semaphores guarantee

- Mutual exclusion
- Freedom from starvation
- Freedom from deadlock

So now it becomes very simple to implement our mutual exclusion that we had earlier, what we do is we just enclose that critical section by proceeding it with a P and following it with a V. So, now if I say P(S) here, then assuming that, that S is 0 or 1, then only one of them will actually see it as one and we will get it. And when you come out of here that one will release it so the other one will get in.

And you can see that the other one is not cooperating, you can go in and come out as many times as you want, because each time you leave, you set it back to one. And if both of them come

there, at the same time, then one is guaranteed to get it because it was one and then it will release it and give it to the other. So, it is not going to be the case that neither of them gets in. And you can actually check that you can generalize this to more than 2, it is not going to actually be limited to.

So, you get mutual exclusion, you get freedoms from starvation, you get freedom from deadlock. So, this really works. And it is much simpler to think about this, as you can see then to think about that turn and request 1 and request 2, that we saw in the Petersons algorithm. So, this is really like a block, we are saying begin the critical section by calling the semaphore with the pass and end the critical section by calling the semaphore with a release.

So, begin get the semaphore release a semaphore and whatever is included between these two is guaranteed to have mutually exclusive access. So, this is what we mean by saying that we want this kind of support from the programming language. So, we cannot program P and V ourselves, it is important to note that it is not possible. I mean, this is a theoretical result, which you can prove that it is not possible to generate an atomic test and set unless it is given to you.

Because whatever you do in a program, any kind of computing environment, this is a two-step process, you have to fetch the value from somewhere look at it and put it back. So, you cannot collapse it into one value unless the hardware provides it to you or the software provides it to you. So, we have to assume that this is available to us, then we can use it to build these critical sections and this mutual exclusive access in a much more simple way for people to use.

(Refer Slide Time: 08:36)

- Too low level
- No clear relationship between a semaphore and the critical region that it protects
- All threads must cooperate to correctly reset semaphore

$P(S)$ if then $\downarrow \downarrow$



- Too low level
- No clear relationship between a semaphore and the critical region that it protects
- All threads must cooperate to correctly reset semaphore
- Cannot enforce that each $P(S)$ has a matching $V(S)$
- Can even execute $V(S)$ without having done $P(S)$



But even though this is better than using, say, Petersons algorithm, it is still a little low level. And the reason it is a little low level is that we have these P's and V's and it is really again the same situation that we have seen in other kinds of coding examples that we have looked at in this course, the question is whether you trust the programmer to do it correctly or not. So, it is only a question of discipline that you put a P before the critical section and a V after the critical section, there is nothing to prevent you from putting a P anywhere or putting a V anywhere.

So there is no clear relationship or no clear requirement that this is the critical section and therefore, these are the semaphores which guard it. So, the connection between the semaphore

and the critical section is left to the programmer to decide and it is not really a part of the code in any sensible way.

The other thing is that it requires everybody still to cooperate. Supposing I do the P and I do not do the V. It is not like a begin end, it is not like a brace, it is not something syntactic and in fact, there will be situations where depending on what you do, you will release a semaphore in different context. So, you might have an if then else. So, you do a P here, and you might do a V separately in the then block and in the else block.

Now, this will be very hard to enforce from a compiler point of view to make sure that every P has a matching V. So, you cannot even. So, first of all, you have to make sure that the programmer always releases the resource that they are blocked. And secondly, the compiler also has to be able to check that no release no block is no pass is actually left with a hanging, deadlock, because there is no corresponding release.

The other thing is even more strange, which is you what happens if you release without having So, again, because it could be conditional execution, there could be some path in which you bypass the P but you execute the V. So, then you will notify people and you will increment a counter which is already free. So, now the value which would have been 0 or 1 and now suddenly become 2.

And now, the whole thing after this will become very unpredictable, because the whole protocol of semaphores is designed to work only if the values are 0 and 1. So, for all these reasons semaphores themselves are not really, even though they provide us with a higher level solution than say, these mutual exclusion protocols that we looked at Peterson and so on, they are still not really at a comfortable level that we would like to work with, in some language like Java.

(Refer Slide Time: 11:15)

- Test-and-set is at the heart of most race conditions
- Need a high level primitive for atomic test-and-set in the programming language
- Semaphores provide one such solution
- Solutions based on test-and-set are low level and prone to programming errors



So, to summarize, test and set is really the heart of the topic. There is no way without being given to you as a primitive to check the value and updated based on what you read in one atomic step. So, we need this high level primitive and semaphores provide us. So, semaphores were introduced by Dijkstra. So, semaphores provide us with one such solution and insignificantly cleans up the code in terms of what we saw before, like the critical section with the Petersons algorithm code, for example.

But still, the semaphores are low level. And so we have these P and V operations which are not closely tied to each other and not even closely tied to what they are protecting. So, we have to really work hard to ensure that discipline programmers use it properly. And this is always a problem that we want to avoid. We want to make sure that it is not up to the programmer to do a discipline, but it is up to the compiler to check the discipline is being followed.

सिद्धिर्भवति कर्मजा